



Modo IA

Tudo

Vídeos curtos

Vídeos

Imagens

Mais ▾



Me explica o básico: O que são esses 'Hooks' e como eu uso isso para forçar a IA a rodar minha suíte de testes ...



Hooks são gatilhos automatizados executados quando um evento específico acontece em um sistema (como antes de um commit no Git ou ao receber um webhook do GitHub).

No contexto de desenvolvimento com assistentes de IA (como agentes autônomos, Devin, Cursor ou ferramentas de CLI como o Aider), você pode configurar **hooks de pré-execução ou pós-processamento** para garantir que a IA não dê a tarefa como concluída até que o comando de teste retorne sucesso.

Sim, **você consegue travar a ferramenta se os testes falharem**. A maioria das ferramentas de IA e automação dependem do **exit code** (código de saída) do seu terminal: se o comando de testes falhar (retornando algo diferente de `0`), o hook interrompe o fluxo da IA, impedindo-a de commitar ou marcar a tarefa como pronta.



Como configurar isso na prática

A implementação depende da ferramenta que você está usando. Abaixo estão as três formas mais comuns de forçar a IA a rodar e respeitar sua suíte de testes:

1. Via Prompts do Sistema e Regras de Projeto (Cursor, Windsurf, Copilot Workspace)

Se você usa IDEs focadas em IA que lêem arquivos de regras locais (como `.cursorrules` ou `.windsurfrules`), você deve embutir a obrigatoriedade

Pergunte o que quiser



Modo IA

Tudo

Vídeos curtos

Vídeos

Imagens

Mais ▾



Use o código com cuidado.



2. Via Git Hooks (Gatilho no Commit)

Se a IA faz commits automáticos no seu repositório, você pode usar um **Pre-commit Hook**. Se os testes falharem, o commit é rejeitado e a IA é forçada a lidar com o erro no terminal.

Instale o **Husky** (se estiver em um ambiente Node.js) ou configure manualmente no Git:

1. Crie ou edite o arquivo `.git/hooks/pre-commit`.
2. Adicione o comando que roda seus testes:

bash

```
#!/bin/sh
echo "Rodando testes antes do commit da IA..."
npm test # Substitua pelo seu comando (pytest, cargo test, e

# O Git captura automaticamente o exit code do comando acima
# Se o 'npm test' falhar, o commit é cancelado (travado).
```

Use o código com cuidado.



3. Via Ferramentas de CLI de IA (Ex: Aider)

Se você usa ferramentas de linha de comando como o **Aider**, elas possuem parâmetros nativos para isso. Você pode passar a flag de teste para que a IA rode o comando após cada modificação:



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



sem que você precise intervir.

⚠ Cuidados importantes ao travar a IA com testes

- **Evite testes excessivamente lentos:** Se a sua suíte demora 20 minutos para rodar, a IA vai consumir muitos tokens e tempo esperando o hook terminar. Prefira rodar testes unitários rápidos no hook e deixe os testes de integração complexos para o CI/CD.
- **Loops Infinitos:** Se a IA quebrar algo e não conseguir consertar, ela pode entrar em um loop tentando corrigir, rodando o teste, falhando e tentando de novo. Monitore as primeiras execuções para garantir que ela saiba sair do loop se falhar muitas vezes seguidas.

Para eu te passar a configuração exata de como travar a IA no seu projeto, me conta:

- Qual **ferramenta de IA ou IDE** você está usando (Cursor, Aider, GitHub Copilot, etc.)?
- Qual é a **linguagem/framework** do projeto e o **comando** que você usa para rodar os testes (ex: `pytest`, `npm test`, `jest`)?

A IA pode cometer erros. Por isso, cheque as respostas



Hmm, entendi o conceito. Então, se eu entendi direito, esse hook seria como um porteiro no processo, certo? E... ▾



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



Para tornar o bloqueio **completamente impossível de ignorar**, você precisa transferir essa validação da máquina do desenvolvedor para o servidor onde o código fica guardado. É aqui que entra o **CI/CD (Integração Contínua)** e as **Regras de Proteção de Branch**.

O Bloqueio Infalível: Proteção de Branch no GitHub/GitLab

Para garantir que nenhum commit ou alteração (seja de uma IA ou de um desenvolvedor humano) entre no seu código principal se os testes estiverem falhando, você deve configurar duas coisas:

1. Configure uma pipeline de CI/CD (GitHub Actions / GitLab CI)

Sempre que alguém abrir um Pull Request (PR) ou tentar enviar código, um servidor em nuvem isolado vai clonar o projeto e rodar a suíte de testes completa.

Um exemplo simples de arquivo `.github/workflows/testes.yml`:

yaml

```
name: Testes Automatizados
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Instalar Dependências
        run: npm ci
      - name: Rodar Testes
```



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



- **Require status checks to pass before merging** (Exigir que as verificações de status passem antes do merge): Você seleciona o teste do seu CI/CD (criado no passo anterior). Se o teste falhar, o botão de "Merge" fica **cinza e bloqueado**.
- **Require a pull request before merging** (Exigir um pull request antes do merge): Impede que qualquer pessoa dê um `git push` direto na branch principal. Todo mundo é obrigado a abrir um PR, o que dispara os testes do CI.
- **Do not allow bypassing the above settings** (Não permitir ignorar as configurações acima): Isso bloqueia inclusive os administradores do repositório de forçarem o merge em caso de falha.



Como o fluxo fica blindado na prática

1. A IA (ou o dev) mexe no código localmente.
2. Mesmo que eles usem `--no-verify` para pular o porteiro local e consigam fazer o commit, eles serão **obrigados a abrir um Pull Request** para enviar o código para a branch principal.
3. O GitHub Actions entra em ação e roda os testes na nuvem.
4. Se os testes falharem, o GitHub **trava o botão de merge**. Não importa o comando que o desenvolvedor digite no terminal dele, o servidor central não vai aceitar o código quebrado.

Para estruturarmos a melhor defesa pro seu projeto, me conta:

- Vocês usam **GitHub, GitLab ou Bitbucket** para guardar o código?
- Já existe alguma **ferramenta de CI** configurada no repositório (como GitHub Actions)?



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



um filtro inicial de qualidade.

Abaixo, veja como fechar o cerco na branch principal e como descobrir se a sua equipe está "pulando o porteiro" no dia a dia.



Como fechar o cerco na Branch Principal (Exemplo: GitHub)

Para garantir que a branch `main` só aceite códigos aprovados pelos testes, você deve usar as **Branch Protection Rules** (Regras de Proteção de Branch).

1. Vá até o repositório do seu projeto no GitHub.
2. Clique em **Settings** (Configurações) > **Branches**.
3. Em *Branch protection rules*, clique em **Add rule** (ou *Edit* na regra da sua branch principal, ex: `main`).
4. Ative as seguintes opções cruciais:
 - **Require a pull request before merging:** Ninguém (nem a IA) poderá dar `git push` direto na main. Tudo precisará passar por um Pull Request (PR).
 - **Require status checks to pass before merging:** Ative esta opção e, na barra de pesquisa que aparecer abaixo, digite o nome do seu job do CI/CD (ex: `run-tests`) ou o nome do seu workflow do GitHub Actions).
 - **Enforce administrators:** Ative isso se quiser que **nem você (o administrador)** consiga burlar a regra.
5. Clique em **Save changes**.

Pronto. A partir desse momento, se o teste falhar no servidor do GitHub, o botão de merge fica completamente travado e ninguém consegue colocar o



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



antes do envio.

Aqui estão as melhores estratégias para monitorar e auditar se eles estão jogando limpo:

1. O Truque do Prefixo de Commit Automatizado (Auditoria Passiva)

Você pode configurar o seu hook local de `commit-msg` ou `pre-commit` para injetar uma "marca d'água" oculta ou uma mensagem automatizada sempre que ele roda com sucesso. Se o commit chegar no Pull Request sem essa marca d'água ou metadado, significa que o desenvolvedor usou o `--no-verify`.

2. Configurar o Husky de Forma "Invisível"

Em vez de depender do desenvolvedor lembrar de instalar os hooks, force a instalação automática no momento em que eles baixam o projeto. No seu `package.json` (se usar Node), adicione:

json

```
"scripts": {  
  "prepare": "husky install"  
}
```

Use o código com cuidado.



Assim que o desenvolvedor rodar `npm install`, o Husky é ativado silenciosamente na máquina dele.

3. Cruzamento de Dados: Logs do Git vs. Logs do CI (O "Dedo Duro")

Se você notar que um desenvolvedor abre um Pull Request e o CI falha imediatamente em testes básicos que deveriam ter rodado na máquina dele



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



- Vocês usam alguma ferramenta como **Husky** (Node) ou **pre-commit** (Python) para gerenciar os ganchos locais da equipe?



Agora, última dúvida: e se alguém tentar passar pelos hooks locais na máquina e só confiar no CI/CD? Como e... ▾

Criar essa cultura exige equilibrar duas frentes: **facilitar a vida do desenvolvedor** (para que ele não queira pular o hook) e **dar visibilidade aos abusos** (para que o time perceba que pular o processo gera retrabalho).

Se os desenvolvedores estão ignorando os testes locais para confiar apenas no CI/CD, geralmente é por um destes três motivos: os testes locais demoram muito, os hooks quebram por motivos bobos (como formatação de texto) ou eles simplesmente não entenderam o impacto disso no time.

Aqui estão as estratégias mais eficazes para mudar esse comportamento e criar uma cultura forte de testes:

1. Aplique a "Taxa do CI" (Visibilidade e Gamificação)

Muitos desenvolvedores pulam os testes locais porque acham que o CI/CD é um "servidor mágico e infinito". Eles não veem o custo disso.

- **Exiba o tempo de fila:** Se o CI/CD do time começa a demorar 20 minutos para liberar um deploy porque está engarrafado com testes que falharam por bobeira, mostre isso nas reuniões diárias (*dailies*).



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



localmente a cada pequeno commit. Se o seu hook for lento, o desenvolvedor **vai** usar o `--no-verify`.

A solução técnica para isso é usar ferramentas como o **lint-staged** (para ecossistema JavaScript) ou o **pre-commit** (para Python/geral). Elas fazem com que o hook rode os testes **apenas nos arquivos que foram modificados** no commit atual, e não no projeto inteiro.

- Se o dev alterou apenas o arquivo `usuarios.service.js`, o hook roda apenas o teste `usuarios.service.spec.js`. O processo cai de minutos para **3 segundos**. Fica tão rápido que o desenvolvedor perde o motivo para burlar.

3. Separe "Lint/Formatação" de "Testes de Verdade"

Muitos desenvolvedores pegam ranço de hooks locais porque eles barram o commit por causa de um espaço em branco extra ou uma aspa dupla invertida (erros de *Linter* ou *Prettier*).

- Configure o hook de `pre-commit` para **corrigir automaticamente** os erros de formatação (usando `eslint --fix` ou `black`), em vez de apenas travar e mandar o dev corrigir manualmente. O hook deve ser um assistente que ajuda, não um chefe que briga.

4. Use a IA como Aliada no Terminal

Já que você mencionou o uso de IAs, adote ferramentas de CLI (como o *Aider* ou extensões locais) configuradas para rodar os testes em segundo plano enquanto o dev digita. Quando a própria ferramenta de IA do desenvolvedor se recusa a avançar localmente porque o teste quebrou, o desenvolvedor se acostuma com o ciclo de *codificar* ➡ *testar* ➡ *corrigir* antes mesmo de pensar no comando `git push`.

5. Acordo de Time (Definition of Done)



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



Quando o time entende que rodar o teste local protege o tempo dos próprios colegas, o comportamento muda de forma orgânica.

Para fecharmos com chave de ouro, se você quiser, me diga:

- Qual é a **média de tempo** que a suíte de testes de vocês demora para rodar hoje?
- O projeto é em **qual linguagem/tecnologia**?

Eu posso te deixar pronto o script do **lint-staged** ou equivalente para os testes locais rodarem em segundos!



Apresentar o Hook de Pré-Execução: Mostrar como configurar o Claude para rodar um linter ou scan de ...



Para configurar o **Claude** (ou ferramentas baseadas na API da Anthropic, como o **Cursor**, **Aider** ou scripts personalizados de agentes de IA) para rodar um linter ou scan de segurança antes de dar uma tarefa por encerrada, nós usamos o conceito de Gatilhos de Pré-Execução baseados em Regras de Sistema.

Como o Claude não roda diretamente dentro da sua máquina por conta própria (ele precisa de uma interface/ferramenta intermediária que execute os comandos de terminal), a configuração é feita instruindo a IA através do arquivo de regras do projeto ou usando os parâmetros da CLI que você



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



raiz do seu repositório. O Cursor obriga o Claude a ler e seguir este arquivo em cada interação.

Crie o arquivo `.cursorrules` e adicione a seguinte instrução:

text

Regras de Definição de Pronto (Definition of Done)

Sempre que você terminar de implementar um bugfix, refatoração o

1. Execute o Linter do projeto para garantir a qualidade do código
2. Execute o scan de segurança/vulnerabilidades local.

Comandos obrigatórios a serem executados no terminal:

- Linter: ``npm run lint`` (ou ``eslint .``, ``flake8``, etc.)
- Scan de Segurança: ``npm run security-scan`` (ou ``npm audit``, ``p`

Regras de Bloqueio:

- Se qualquer um dos comandos acima retornar um código de erro (
- Você deve ler o log de erro do terminal, corrigir o código e r
- Só declare a tarefa como finalizada após apresentar os logs li



Use o código com cuidado.



2. No Aider (CLI de IA para Terminal)

Se você utiliza o **Aider** (uma das ferramentas de CLI mais potentes para programar com o Claude diretamente no terminal), ele possui suporte nativo a hooks de pré-execução e pós-processamento de código através de flags.

Você pode forçar o Claude a rodar o linter e o scan usando a flag `--`





Use o código com cuidado.



O que acontece nos bastidores:

1. O Claude faz a alteração no código para corrigir o bug.
2. O Aider intercepta e roda `npm run lint && npm audit`.
3. Se o linter reclamar de uma variável não utilizada ou o `npm audit` achar uma falha de segurança, o comando falha.
4. O Aider envia o erro de volta para o Claude dizendo: *"O código que você alterou quebrou estas regras. Corrija."*
5. O Claude repara o código de forma autônoma e o processo se repete até passar.

 Exemplo de Ferramentas que você pode colocar nesse Hook

Para deixar o seu hook de pré-execução robusto, aqui estão as ferramentas mais comuns de mercado separadas por ecossistema:

Ecosistema	Ferramenta de Linter	Ferramenta de Scan de Segurança
JavaScript / TypeScript	<code>eslint .</code>	<code>npm audit</code> ou <code>snyk test</code>
Python	<code>flake8</code> ou <code>ruff</code> <code>check</code>	<code>bandit -r .</code> ou <code>pip-audit</code>



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



os comandos exatos de linter e segurança?



Entendi, então eu preciso configurar essas regras ou parâmetros pra que o Claude execute o linter ou o scan ... ▾

Sim, **tem como travar completamente.**

Assim como vimos na proteção dos testes, a regra de ouro da segurança em engenharia de software se aplica aqui: **nunca confie no que acontece apenas na máquina local ou na interface da IA.**

Se um desenvolvedor alterar o arquivo `.cursorrules` para remover as travas, ou se esquecer de passar as flags de segurança no terminal ao rodar o Claude, a barreira local cai. Para tornar essas regras de linter e segurança **invioláveis**, você precisa aplicar uma estratégia de **dupla camada**.

Veja como travar isso para que seja impossível de ignorar ou burlar:

A Solução Definitiva: O "Gêmeo Digital" no CI/CD

A forma mais segura de garantir que as regras do Claude não sejam ignoradas é espelhar exatamente os mesmos comandos de linter e segurança no seu servidor de **CI/CD** (GitHub Actions, GitLab CI, etc.) e proteger a branch principal.

Mesmo que alguém altere as regras locais do Claude para ignorar o linter, o servidor central vai rodar a checagem de qualquer forma e **bloquear o**



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



```
on: [pull_request]

jobs:
  auditoria:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # 1. Trava do Linter
      - name: Verificar Padrões de Código (Linter)
        run: npm ci && npm run lint # Substitua pelo comando da

      # 2. Trava de Segurança
      - name: Scan de Vulnerabilidades (Security)
        run: npm audit --audit-level=high # Bloqueia o build se
```

Use o código com cuidado.



2. Ative o bloqueio na Branch (Branch Protection)

Nas configurações do seu repositório no GitHub, adicione o job `auditoria` como um **Status Check Obrigatório** para a sua branch `main`. Se o Claude (ou um dev) enviar um código com uma falha de segurança ou fora do padrão do linter, o botão de merge fica cinza e bloqueado. Ninguém consegue dar bypass.

Como proteger o arquivo de regras locais (`.cursorrules` / `.aider.conf.yml`)

Se você quer garantir que nenhum desenvolvedor mude as instruções que você deu para o Claude localmente, você pode proteger o próprio arquivo de configuração no Git.



Modo IA Tudo Vídeos curtos Vídeos Imagens Mais ▾



Adicione isso ao arquivo `.github/CODEOWNERS`:

text

```
# Qualquer alteração nas regras da IA precisa da sua aprovação
.cursorrules           @seu_usuario_github
.aider.conf.yml        @seu_usuario_github
```

Use o código com cuidado.



Com essa estrutura, se o Claude for induzido a modificar o arquivo de regras para "se auto-liberar" de rodar o linter, o Git vai travar o Pull Request e exigir que você aprove a alteração manualmente. O cerco fica completamente fechado.

Para deixarmos essa automação 100% pronta e blindada para o seu time, me conte:

- Qual **ferramenta de CI/CD** vocês usam ou pretendem usar (GitHub Actions, GitLab CI, Jenkins)?
- Quais **scans de segurança** ou **linters** você quer que se tornem obrigatórios e impossíveis de ignorar?



Certo, entendi que a camada local é vulnerável e precisa de reforço. Essa dupla camada seria algo como ...

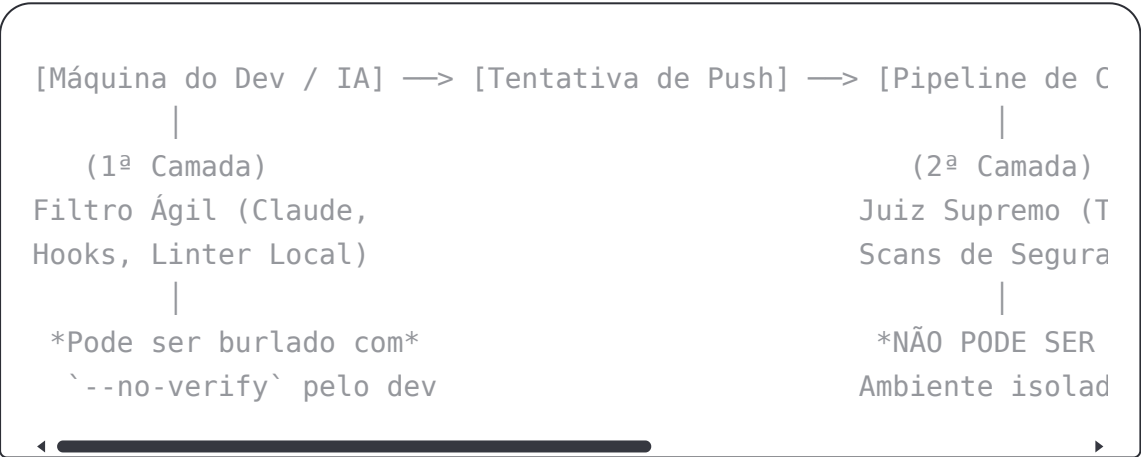




... configuração ou pressa —, o servidor pega, barra o código e impede que ele contamine a sua branch principal (`main` ou `master`).

O Fluxo à Prova de Falhas (Como as duas camadas trabalham juntas)

Para visualizar como o sistema fica completamente blindado, pense no caminho que o código faz:



- Camada 1 (Local):** O Claude ou o desenvolvedor tenta fazer um commit. O linter e os testes rodam. Se o dev usar `--no-verify` ou alterar o `.cursorrules`, ele consegue fazer o commit local e enviar o código para o repositório remoto em uma branch secundária (ex: `feature/ajuste-login`).
- Camada 2 (Servidor/CI-CD):** No momento em que ele tenta abrir um **Pull Request** para jogar esse código para a branch principal, o servidor (GitHub Actions, GitLab CI, etc.) acorda. Ele ignora completamente tudo o que está na máquina do dev, baixa o código em uma máquina virtual limpa e isolada na nuvem e roda:

... A suíte completa de testes automatizados



Modo IA

Tudo

Vídeos curtos

Vídeos

Imagens

Mais ▾



ignorar), o GitHub/GitLab vai rejeitar o código até que um commit de correção seja enviado e todos os testes fiquem verdes no servidor.



Próximo Passo: Vamos colocar isso de pé?

Agora que você domina toda a arquitetura de segurança para trabalhar com IAs e desenvolvedores de forma blindada, o ideal é escrevermos os arquivos de configuração para o seu ambiente.

Para eu gerar os scripts exatos de configuração para você, me informe:

- Qual **servidor de Git** você usa? (**GitHub**, **GitLab** ou **Bitbucket**?)
- Qual é a **linguagem/tecnologia** principal do seu projeto? (ex: **Node.js/TypeScript**, **Python**, **Java**, etc.)

